



C Piscine

C 01

Summary: This document is the subject for the module C 01 of the C Piscine @ 42.

Version: 5

Contents

I	Instructions	2
II	Foreword	4
III	Exercise 00 : <code>ft_ft</code>	5
IV	Exercise 01 : <code>ft_ultimate_ft</code>	6
V	Exercise 02 : <code>ft_swap</code>	7
VI	Exercise 03 : <code>ft_div_mod</code>	8
VII	Exercise 04 : <code>ft_ultimate_div_mod</code>	9
VIII	Exercise 05 : <code>ft_putstr</code>	10
IX	Exercise 06 : <code>ft_strlen</code>	11
X	Exercise 07 : <code>ft_rev_int_tab</code>	12
XI	Exercise 08 : <code>ft_sort_int_tab</code>	13
XII	Submission and peer-evaluation	14

Chapter I

Instructions

- Only this page will serve as reference: do not trust rumors.
- Watch out! This document could potentially change up before submission.
- Make sure you have the appropriate permissions on your files and directories.
- You have to follow the submission procedures for all your exercises.
- Your exercises will be checked and graded by your fellow classmates.
- On top of that, your exercises will be checked and graded by a program called Moulinette.
- Moulinette is very meticulous and strict in its evaluation of your work. It is entirely automated and there is no way to negotiate with it. So if you want to avoid bad surprises, be as thorough as possible.
- Moulinette is not very open-minded. It won't try and understand your code if it doesn't respect the Norm. Moulinette relies on a program called `norminette` to check if your files respect the norm. TL;DR: it would be idiotic to submit a piece of work that doesn't pass `norminette`'s check.
- These exercises are carefully laid out by order of difficulty - from easiest to hardest. We **will not** take into account a successfully completed harder exercise if an easier one is not perfectly functional.
- Using a forbidden function is considered cheating. Cheaters get **-42**, and this grade is non-negotiable.
- You'll only have to submit a `main()` function if we ask for a program.
- Moulinette compiles with these flags: `-Wall -Wextra -Werror`, and uses `gcc`.
- If your program doesn't compile, you'll get 0.
- You cannot leave any additional file in your directory than those specified in the subject.
- Got a question? Ask your peer on the right. Otherwise, try your peer on the left.

- Your reference guide is called `Google / man / the Internet / ....`
- Check out the "C Piscine" part of the forum on the intranet, or the slack Piscine.
- Examine the examples thoroughly. They could very well call for details that are not explicitly mentioned in the subject...
- By Odin, by Thor ! Use your brain !!!



Norminette must be launched with the `-R CheckForbiddenSourceHeader` flag. Moulinette will use it too.

Chapter II

Foreword

Vincent: And you know what they call a... a... a Quarter Pounder with Cheese in Paris?

Jules: They don't call it a Quarter Pounder with cheese?

Vincent: No man, they got the metric system. They wouldn't know what the fuck a Quarter Pounder is.

Jules: Then what do they call it?

Vincent: They call it a Royale with cheese.

Jules: A Royale with cheese. What do they call a Big Mac?

Vincent: Well, a Big Mac's a Big Mac, but they call it le Big-Mac.

Jules: Le Big-Mac. Ha ha ha ha. What do they call a Whopper?

Vincent: I dunno, I didn't go into Burger King.

At least one of the following exercises has nothing to do you with a Royale with cheese.

Chapter III

Exercise 00 : **ft_ft**

	Exercise 00
ft_ft	
Turn-in directory : <i>ex00/</i>	
Files to turn in : ft_ft.c	
Allowed functions : None	

- Create a function that takes a pointer to int as a parameter, and sets the value "42" to that int.
- Here's how it should be prototyped :

```
void ft_ft(int *nbr);
```

Chapter IV

Exercise 01 : ft_ultimate_ft

	Exercise 01
ft_ultimate_ft	
Turn-in directory : <i>ex01/</i>	
Files to turn in : ft_ultimate_ft.c	
Allowed functions : None	

- Create a function that takes a pointer to pointer to pointer to pointer to pointer to pointer to pointer to pointer to int as a parameter and sets the value "42" to that int.
- Here's how it should be prototyped :

```
void      ft_ultimate_ft(int *****nbr);
```

Chapter V

Exercise 02 : **ft_swap**

	Exercise 02
	ft_swap
	Turn-in directory : <i>ex02/</i>
	Files to turn in : ft_swap.c
	Allowed functions : None

- Create a function that **swaps** the value of two integers **whose addresses are entered as parameters**.
- Here's how it should be prototyped :

```
void    ft_swap(int *a, int *b);
```


Chapter VI

Exercise 03 : **ft_div_mod**

	Exercise 03
	ft_div_mod
Turn-in directory : <i>ex03/</i>	
Files to turn in : ft_div_mod.c	
Allowed functions : None	

- Create a function `ft_div_mod` prototyped like this :

```
void    ft_div_mod(int a, int b, int *div, int *mod);
```

- This function divides parameters `a` by `b` and stores the result in the `int` pointed by `div`. It also stores the remainder of the division of `a` by `b` in the `int` pointed by `mod`.

Chapter VII

Exercise 04 : `ft_ultimate_div_mod`

	Exercise 04
<code>ft_ultimate_div_mod</code>	
Turn-in directory : <i>ex04/</i>	
Files to turn in : <code>ft_ultimate_div_mod.c</code>	
Allowed functions : None	

- Create a function `ft_ultimate_div_mod` with the following prototype :

```
void    ft_ultimate_div_mod(int *a, int *b);
```

- This function divides parameters `a` by `b`. The result of this division is stored in the `int` pointed by `a`. The remainder of the division is stored in the `int` pointed by `b`.

Chapter VIII

Exercise 05 : **ft_putstr**

	Exercise 05
ft_putstr	
Turn-in directory : <i>ex05/</i>	
Files to turn in : ft_putstr.c	
Allowed functions : write	

- Create a function that displays a string of characters on the standard output.
- Here's how it should be prototyped :

```
void    ft_putstr(char *str);
```

Chapter IX

Exercise 06 : **ft_strlen**

	Exercise 06
ft_strlen	
Turn-in directory : <i>ex06/</i>	
Files to turn in : ft_strlen.c	
Allowed functions : None	

- Create a function that counts and returns the number of characters in a string.
- Here's how it should be prototyped :

```
int ft_strlen(char *str);
```

Chapter X

Exercise 07 : **ft_rev_int_tab**

	Exercise 07
	ft_rev_int_tab
	Turn-in directory : <i>ex07/</i>
	Files to turn in : ft_rev_int_tab.c
	Allowed functions : None

- Create a function which reverses a given array of integer (first goes last, etc).
- The arguments are a pointer to int and the number of ints in the array.
- Here's how it should be prototyped :

```
void ft_rev_int_tab(int *tab, int size);
```

Chapter XI

Exercise 08 : ft_sort_int_tab

	Exercise 08
	ft_sort_int_tab
Turn-in directory : <i>ex08/</i>	
Files to turn in : ft_sort_int_tab.c	
Allowed functions : None	

- Create a function which sorts an array of integers by ascending order.
- The arguments are a pointer to int and the number of ints in the array.
- Here's how it should be prototyped :

```
void ft_sort_int_tab(int *tab, int size);
```

Chapter XII

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double check the names of your files to ensure they are correct.

As these assignments are not verified by a program, feel free to organize your files as you wish, as long as you turn in the mandatory files and comply with the requirements.



You need to return only the files requested by the subject of this project.